

EXP 2 MULTI-LAYER PERCEPTRON (MLP) FOR CLASSIFICATION

Aim:

To implement a Multi-Layer Perceptron (MLP) for classifying Iris flower species using different hidden layers and activation functions.

Algorithm:

1. Import the required Python libraries.
2. Load the Iris dataset.
3. Separate input features and target classes.
4. Convert the target labels into one-hot encoded values.
5. Standardize the input features using `StandardScaler`.
6. Split the data into training and testing sets.
7. Create an MLP with different hidden-layer configurations.
8. Train the models using different activation functions such as ReLU, Tanh, and Sigmoid.
9. Evaluate each model using accuracy and loss.
10. Plot the training/validation accuracy and loss, and predict the class of a new Iris sample.

Program:

```
import matplotlib.pyplot as plt

import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense


iris = load_iris()

X = iris.data

y = iris.target.reshape(-1, 1)

classes = iris.target_names
```

```
encoder = OneHotEncoder(sparse_output=False)
```

```
y_encoded = encoder.fit_transform(y)
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y_encoded, test_size=0.2, random_state=42  
)
```

```
def create_mlp(input_dim, output_dim, hidden_layers, activation):
```

```
    model = Sequential()
```

```
    model.add(Dense(hidden_layers[0], input_dim=input_dim,  
                    activation=activation))
```

```
    for units in hidden_layers[1:]:
```

```
        model.add(Dense(units, activation=activation))
```

```
    model.add(Dense(output_dim, activation="softmax"))
```

```
    model.compile(  
        optimizer="adam",
```

```
        loss="categorical_crossentropy",
```

```
        metrics=["accuracy"]
```

```
    )
```

```
    return model
```

```
configs = [[8], [16, 8], [32, 16, 8]]
```

```
activations = ["relu", "tanh", "sigmoid"]
```

```
for hidden_layers in configs:
```

```
    for activation in activations:
```

```
        print("\nHidden Layers:", hidden_layers,  
              "Activation:", activation)
```

```
        model = create_mlp(4, 3, hidden_layers, activation)
```

```
        history = model.fit(  
            X_train, y_train,  
            epochs=50,  
            batch_size=5,  
            verbose=0,  
            validation_split=0.1  
        )
```

```
        loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
```

```
        print("Test Accuracy:", round(accuracy, 4))
```

```
        print("Test Loss:", round(loss, 4))
```

```
# Plot final model performance
```

```
plt.plot(history.history["accuracy"], label="Train Accuracy")
```

```
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
```

```
plt.title("Model Accuracy")
```

```
plt.legend()
```

```
plt.show()
```

```
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Model Loss")
plt.legend()
plt.show()
```

Prediction

```
sepal_length = float(input("Sepal length: "))
sepal_width = float(input("Sepal width: "))
petal_length = float(input("Petal length: "))
petal_width = float(input("Petal width: "))
```

```
user_input = np.array([[
    sepal_length, sepal_width,
    petal_length, petal_width
]])
```

```
user_input_scaled = scaler.transform(user_input)
```

```
prediction = model.predict(user_input_scaled, verbose=0)
```

```
predicted_class = np.argmax(prediction)
```

```
print("Predicted Iris Species:", classes[predicted_class])
```

Output:

```
Hidden Layers: [8] Activation: relu
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the
first layer in the model instead.
```

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Test Accuracy: 1.0
Test Loss: 0.1615

Hidden Layers: [8] Activation: tanh
Test Accuracy: 0.9667
Test Loss: 0.1493

Hidden Layers: [8] Activation: sigmoid
Test Accuracy: 0.9333
Test Loss: 0.4088

Hidden Layers: [16, 8] Activation: relu
Test Accuracy: 1.0
Test Loss: 0.0697

Hidden Layers: [16, 8] Activation: tanh
Test Accuracy: 0.9667
Test Loss: 0.0912

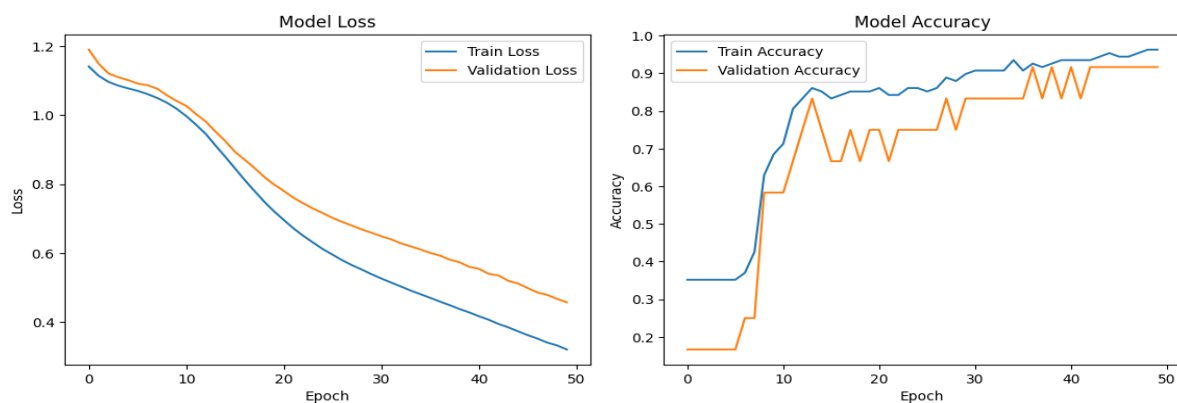
Hidden Layers: [16, 8] Activation: sigmoid
Test Accuracy: 0.9667
Test Loss: 0.4158

Hidden Layers: [32, 16, 8] Activation: relu
Test Accuracy: 1.0
Test Loss: 0.0525

Hidden Layers: [32, 16, 8] Activation: tanh
Test Accuracy: 1.0
Test Loss: 0.0466

Hidden Layers: [32, 16, 8] Activation: sigmoid
Test Accuracy: 0.9333
Test Loss: 0.3127

```



Result

The MLP was successfully implemented using the Iris dataset. Different hidden-layer configurations and activation functions were tested, and the models achieved good classification accuracy. The trained model also successfully predicted the Iris species for a new input.